Lesson 02 Demo 01

Automating RDS Instance Launch Using Terraform and GitOps

Objective: To automate the creation of an Amazon RDS instance using Terraform configuration and a GitOps pipeline powered by GitHub Actions

Tools required: AWS Management Console and GitHub

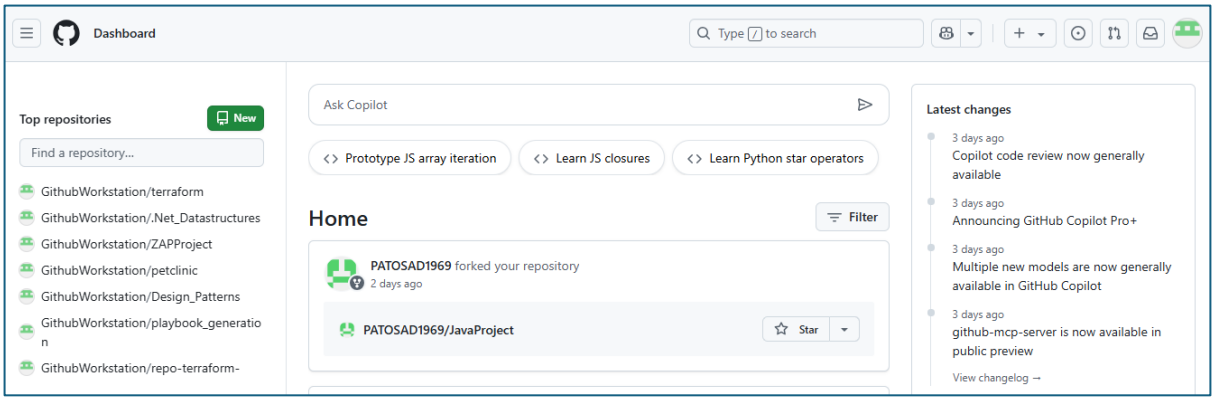
Prerequisites: None

Steps to be followed:

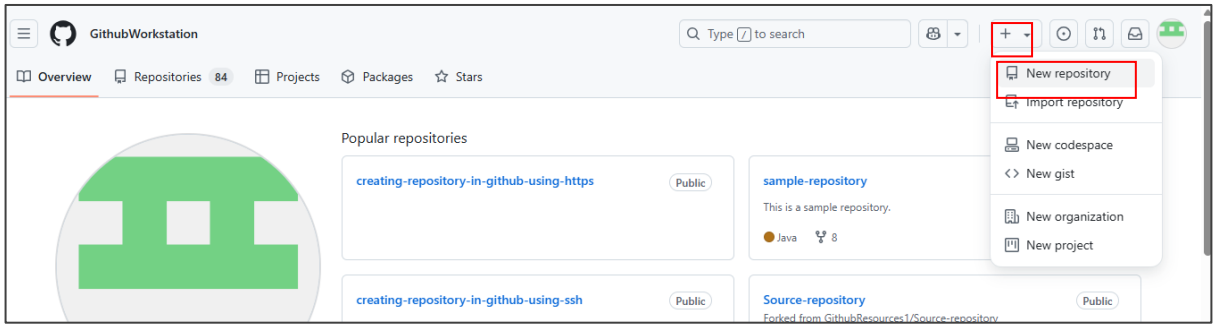
1. Set up Terraform configuration for the RDS instance
2. Build the GitOps pipeline to automate RDS deployment

Step 1: Set up Terraform configuration for the RDS instance

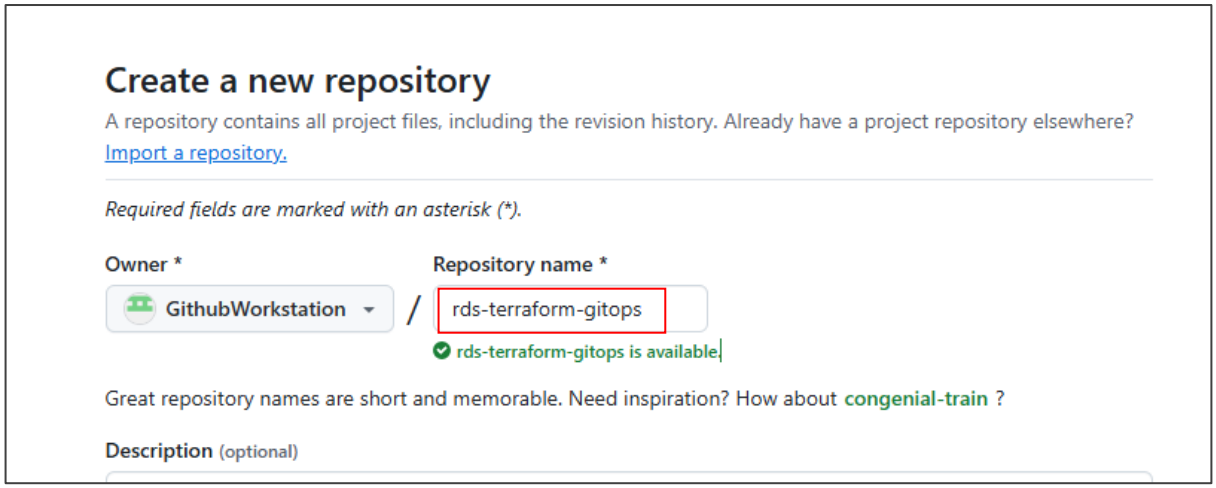
1.1 Sign in to your GitHub account



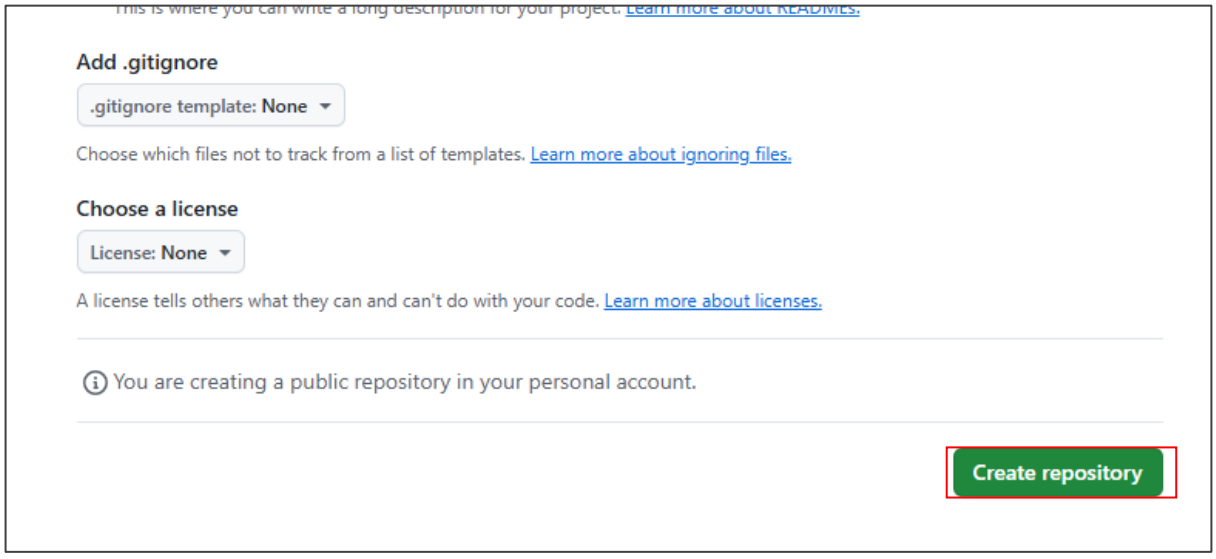
1.2 Click the **plus (+)** icon in the top-right corner and select **New repository**



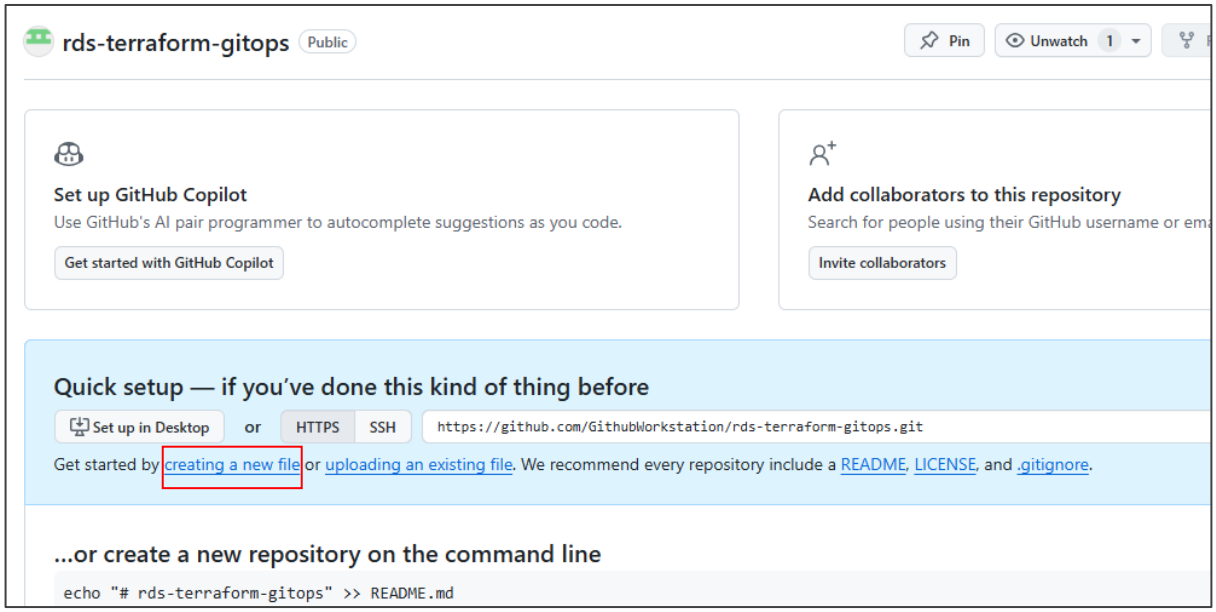
1.3 Enter **rds-terraform-gitops** as the repository name



1.4 Click **Create repository**



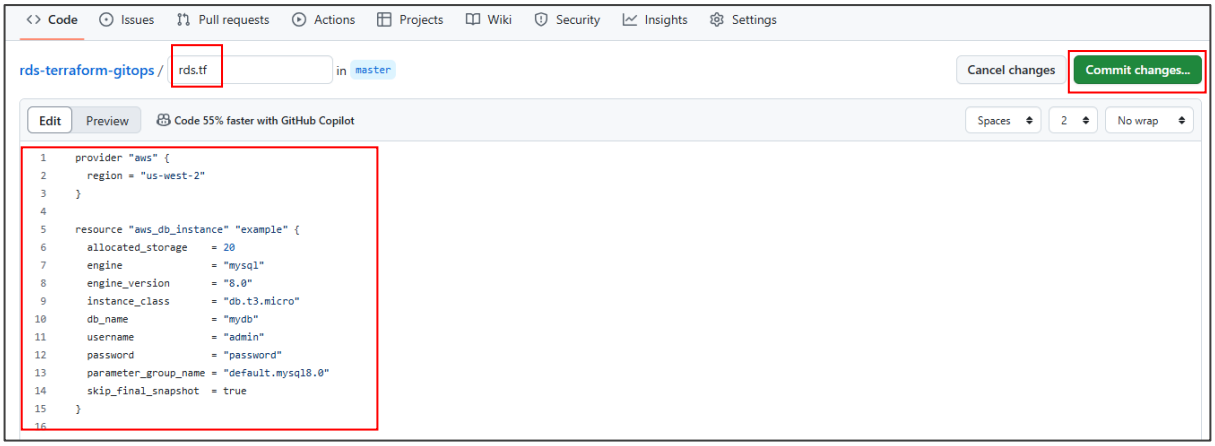
1.5 Under the **Quick setup** section, click **creating a new file**



1.6 Name the file as **rds.tf**, paste the following Terraform configuration code into the editor, and click **Commit changes…** to save it

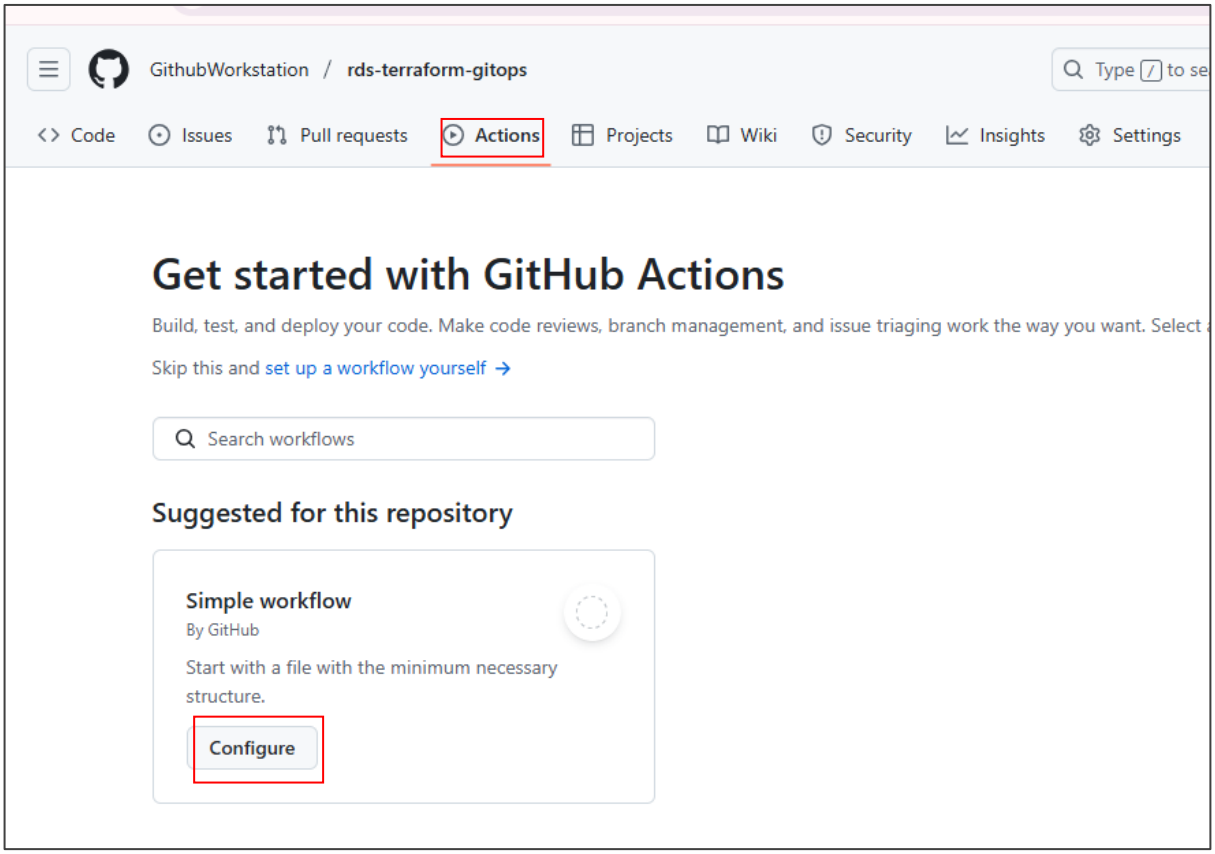
```
provider "aws" {
  region = "us-west-2"
}

resource "aws_db_instance" "example" {
  allocated_storage    = 20
  engine               = "mysql"
  engine_version       = "8.0"
  instance_class       = "db.t3.micro"
  db_name              = "mydb"
  username             = "admin"
  password             = "password"
  parameter_group_name = "default.mysql8.0"
  skip_final_snapshot  = true
}
```



Step 2: Build the GitOps pipeline to automate RDS deployment

2.1 Click the **Actions** tab in your repository and select **Configure** under the **Simple workflow** section



2.2 Name the workflow file as **main.yml**, paste the following code into the editor, and click **Commit changes…**

```
name: terraform

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
jobs:
  terraform:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v2

      - name: Set up Terraform
        uses: hashicorp/setup-terraform@v1
        with:
          terraform_version: 1.0.0

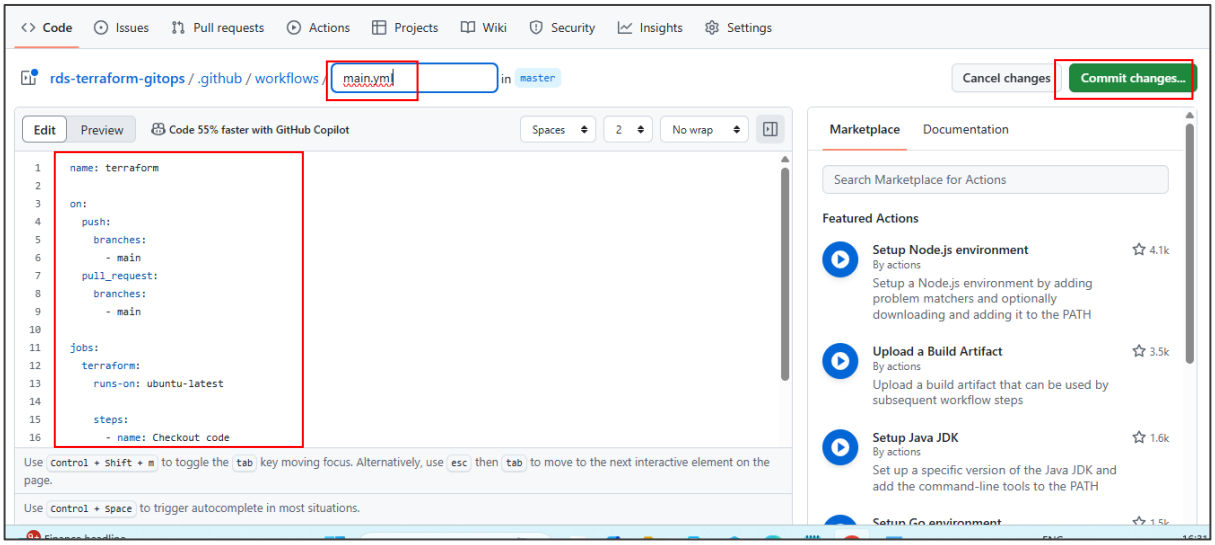
      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v1
        with:
          aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-session-token: "${{ secrets.AWS_ACCESS_TOKEN }}"
          aws-region: us-east-1

      - name: Terraform Init
        run: terraform init

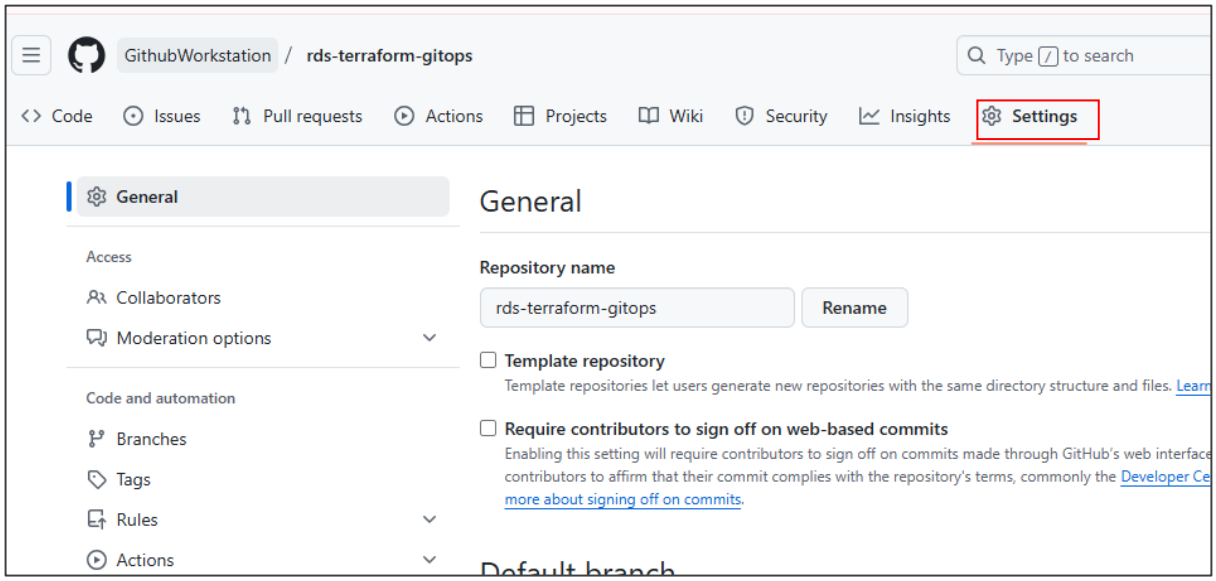
      - name: Terraform Plan
        run: terraform plan

      - name: Terraform Apply
```

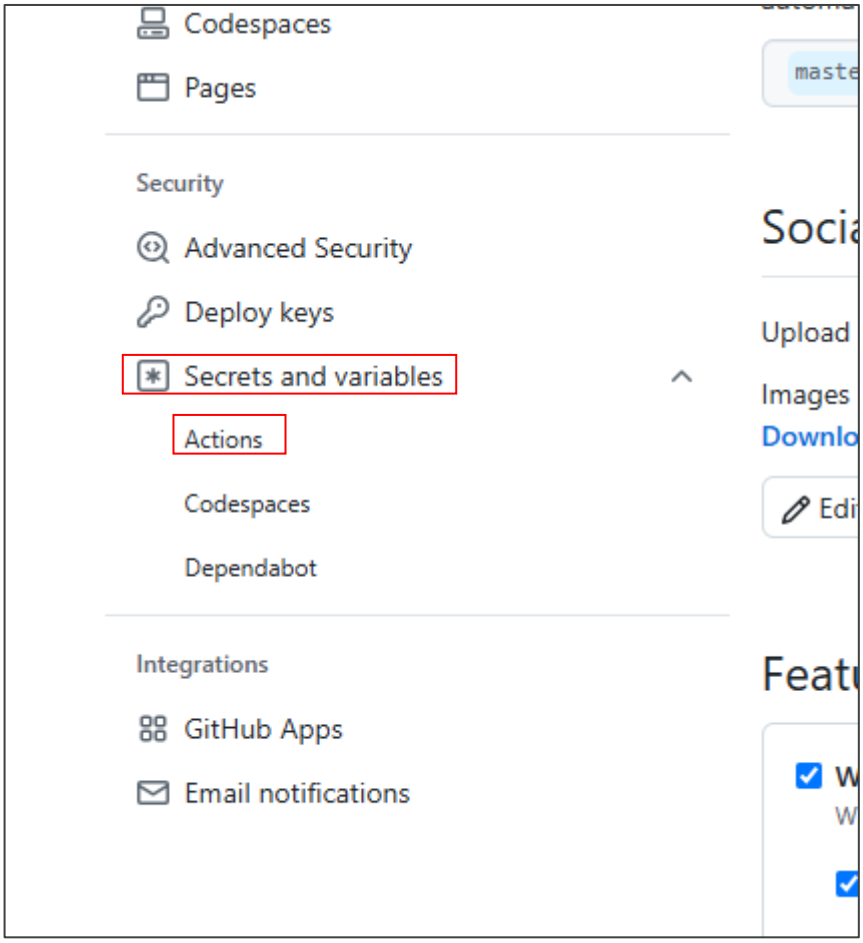
run: terraform apply -auto-approve



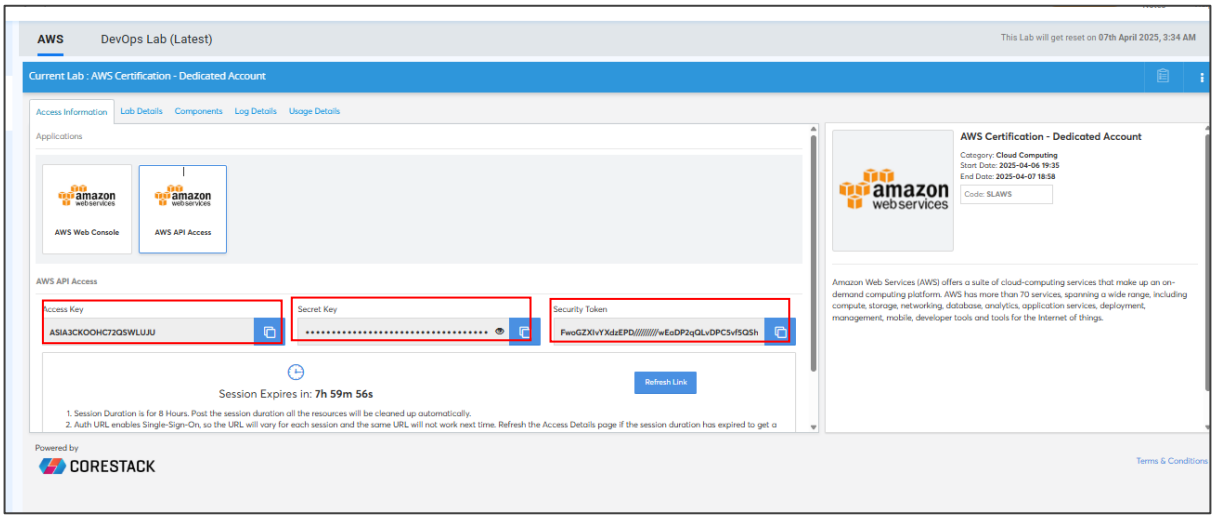
2.3 Go to the **Settings** tab in the top navigation



2.4 Scroll to the **Security** section, click **Secrets and variables**, and then select **Actions**



2.5 From your AWS account, copy the **Access Key**, **Secret Key**, and **Session Token**

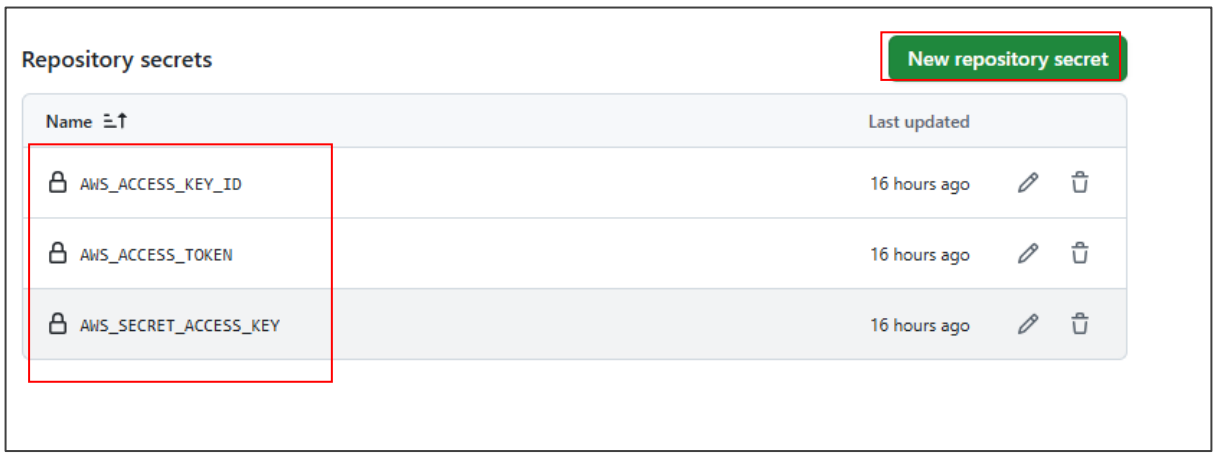


2.6 Click **New repository secret**, then create the following secrets by pasting the copied AWS credentials into the appropriate fields:

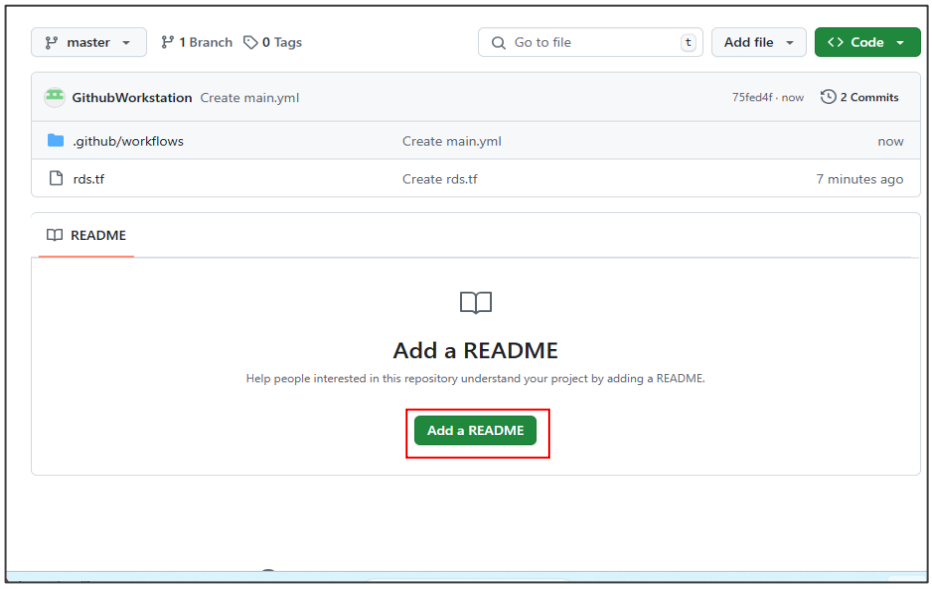
AWS_ACCESS_KEY_ID – Paste the copied Access Key

AWS_ACCESS_TOKEN – Paste the copied Secret Key

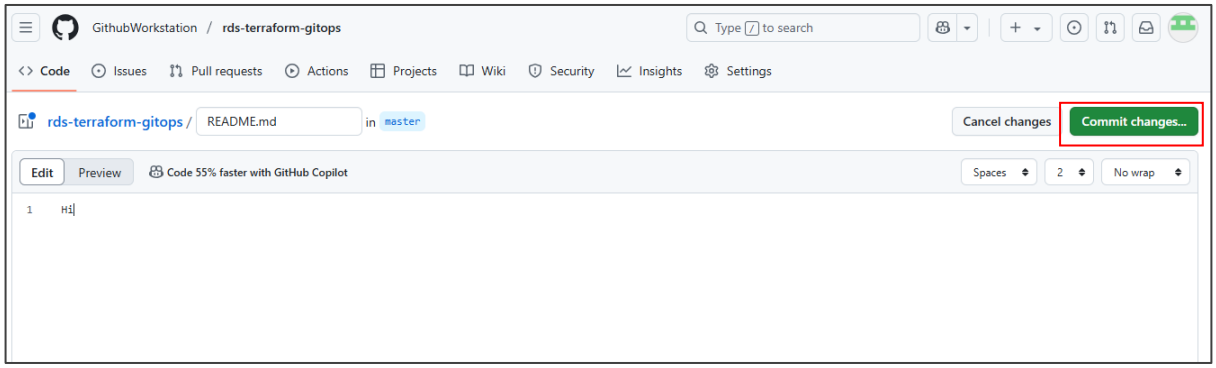
AWS_SECRET_ACCESS_KEY – Paste the copied Security Token



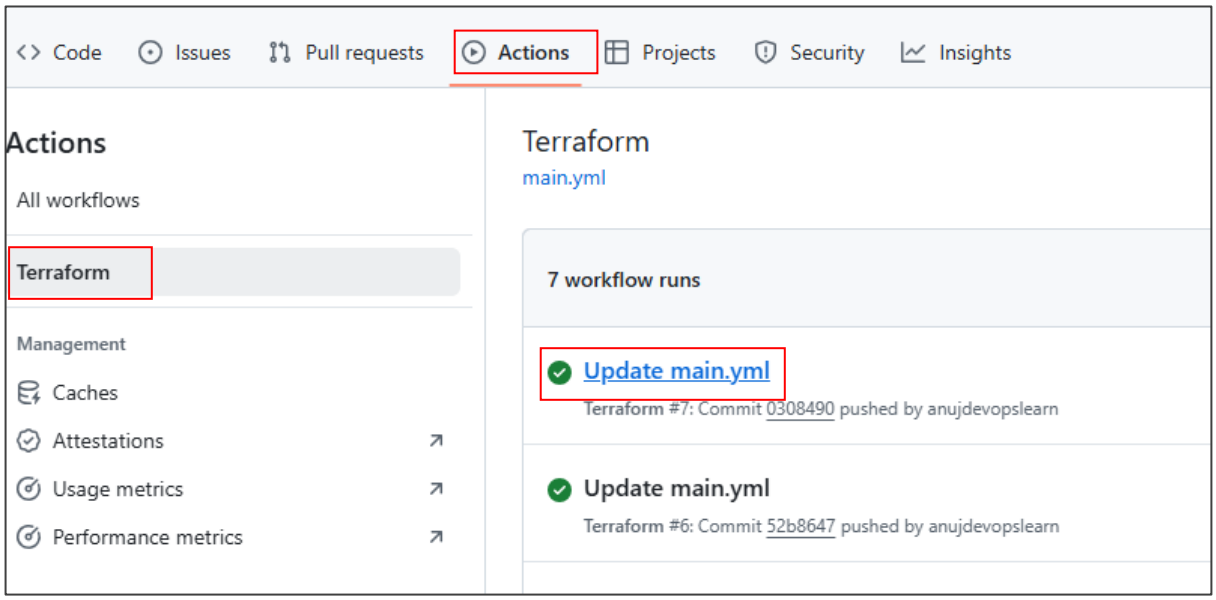
2.7 Return to your repository and click **Add a README**



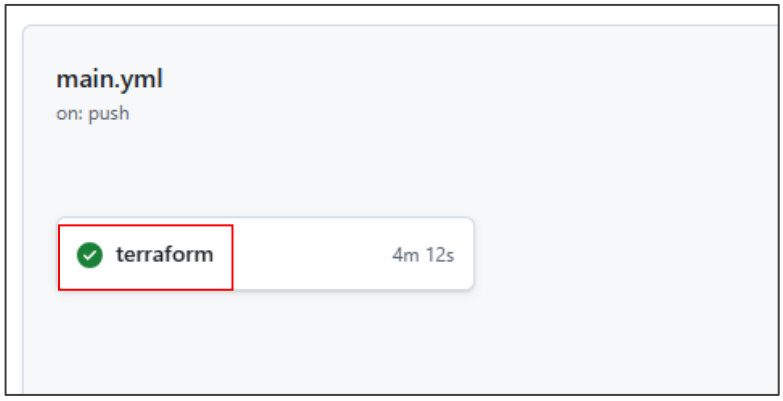
2.8 In the README editor, type a message such as Hi, then click **Commit changes…**



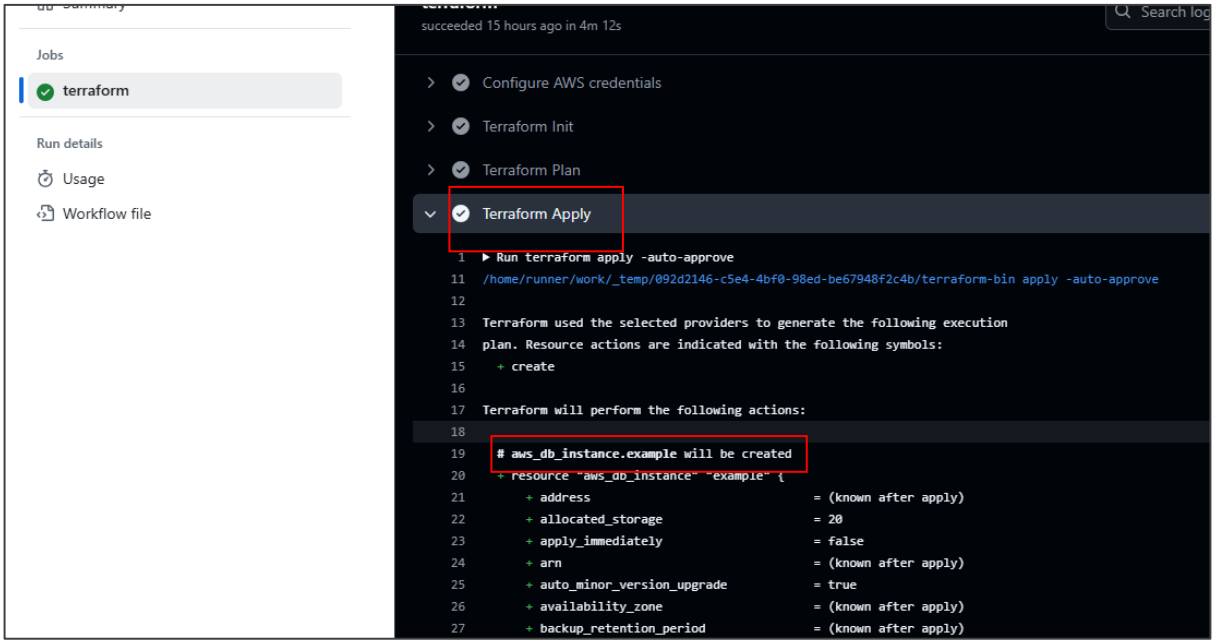
2.9 Navigate to the **Actions** tab, select the **Terraform** workflow, and click **Update main.yml** to view the latest successful run



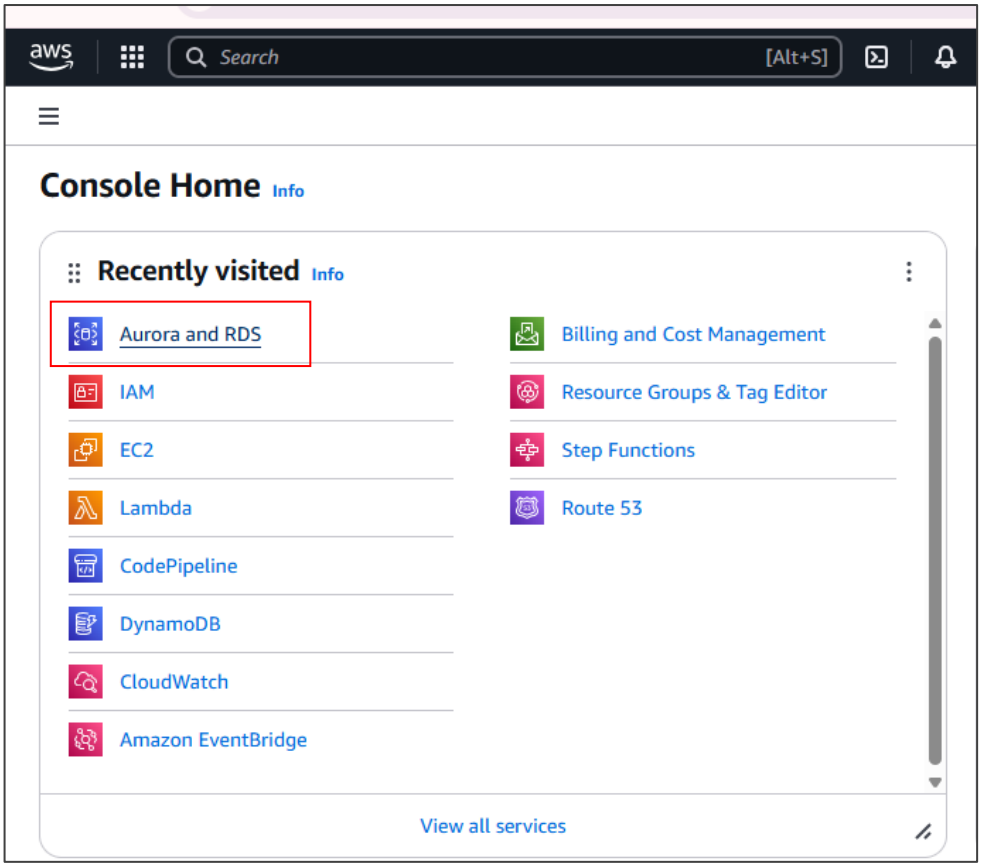
2.10 Click the job box labeled **terraform** to view detailed logs of the workflow



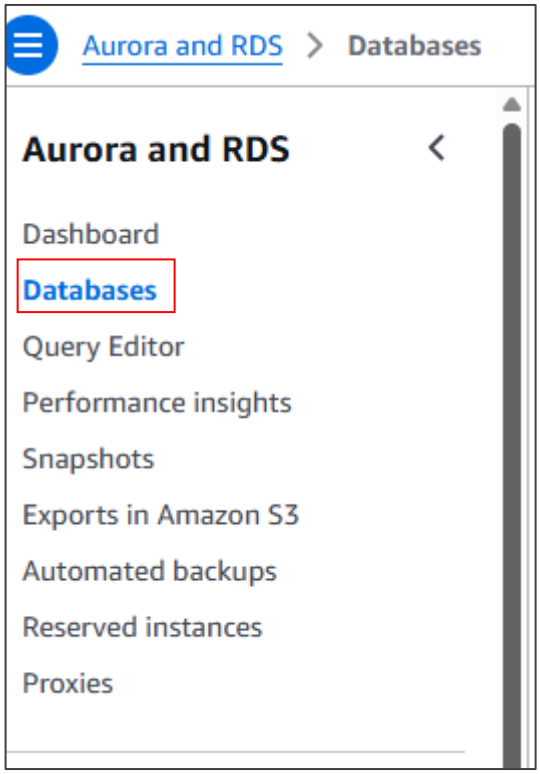
2.11 Expand the **Terraform Apply** section and confirm that the **aws_db_instance.example** resource is listed with the correct attributes



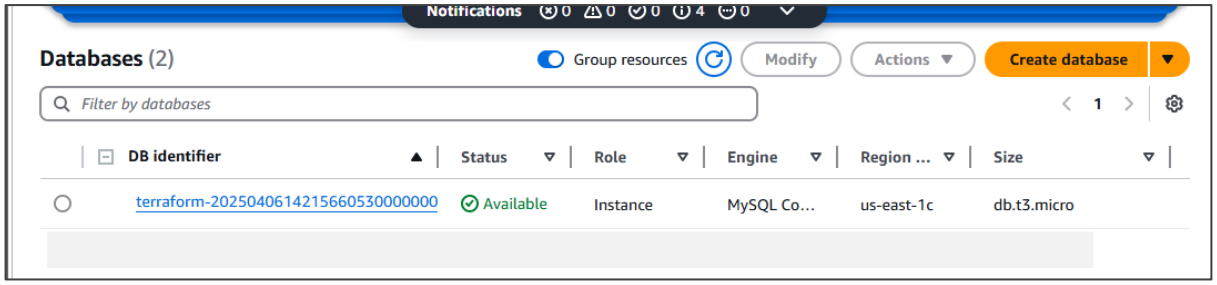
2.12 From the AWS Console Home, click **Aurora and RDS**



2.13 Click **Databases** in the side panel



2.14 Verify that the new database instance is listed with the status **Available** and the instance type **db.t3.micro**



By following these steps, you have successfully implemented a secure, automated GitOps workflow using Terraform and GitHub Actions to provision an Amazon RDS instance.